Лабораторная работа №2

Тема: Создание и настройка кластера Kubernetes

Kubernetes (K8s) — это открытое программное обеспечение для автоматизации развёртывания, масштабирования и управления контейнеризированными приложениями1. Kubernetes группирует контейнеры, составляющие приложение, в логические единицы для более простого управления и обнаружения.

Существуют разнообразные готовые варианты для реализации кластера Kubernetes (K8S).

Например:

- Minikube представляет собой готовый кластер, который можно развернуть на одном компьютере. Это отличный вариант для знакомства с Kubernetes.
- Kubespray — это набор ролей Ansible.
- Также доступны готовые кластеры в облачных сервисах, таких как AWS, Google Cloud, Yandex Cloud и другие.

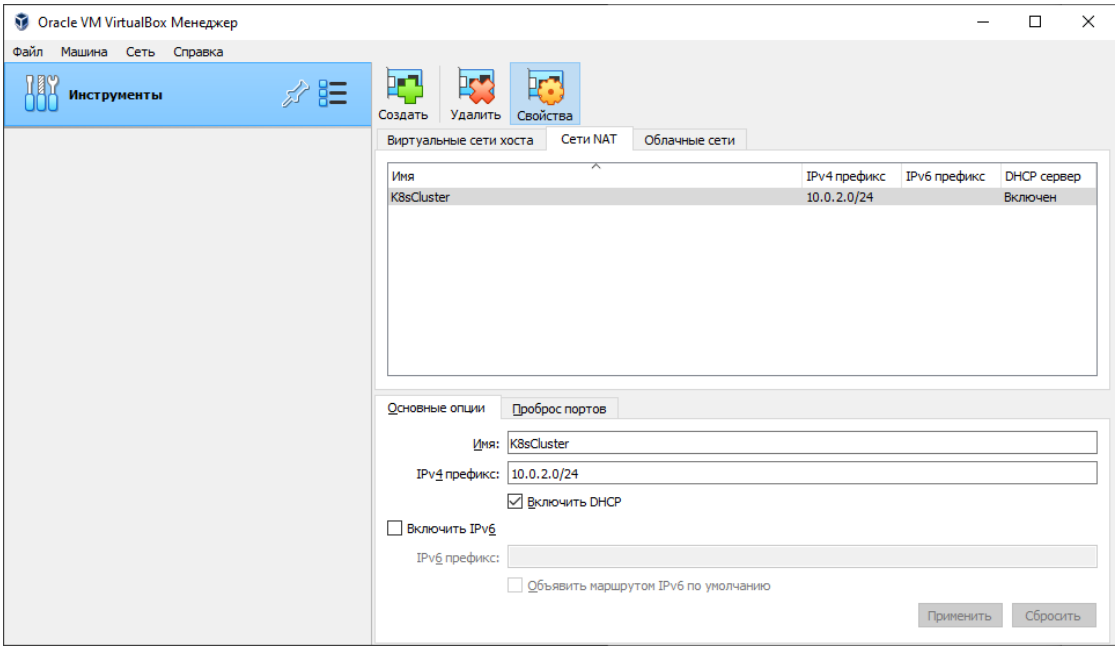
Выбор одной из готовых реализаций является быстрым и надежным способом развертывания системы оркестрации контейнеров Docker. В данной лабораторной работе мы рассмотрим процесс ручного создания кластера Kubernetes, состоящего из трех нод: одной мастер-ноды (для управления) и двух рабочих нод (для запуска контейнеров).

Перед началом работы в качестве мастер-ноды можно использовать ВМ, созданную в Лабораторной работе №0. Для двух рабочих нод нужно будет создать еще две ВМ. **Также необходимо, чтобы в настройках системы каждой ВМ показатель процессоров имел значение 2 и более.**

Все действия в лабораторной работе необходимо сделать на всех 3 ВМ, однако будьте внимательны, в некоторых пунктах есть уточнения, что должно ставиться на конкретную ВМ.

Задание №1 Настройка сети

1. Создайте сеть NAT, чтобы узлы k8s могли взаимодействовать внутри виртуальной среды. Чтобы создать сеть NAT, перейдите в Файл -> Настройки -> Сеть. Создайте новую сеть и определите ей имя. (Пример: K8sCluster)



2. Для определения созданной сети в ВМ, перейдите ВМ -> Настройки -> Сеть -> Адаптер 1. Выберете Тип подключения – сеть NAT и имя созданной ранее сети.

Задание №2 Настройка системы

1. Необходимо задать имена узлам. Для этого выполняем следующую команду на каждой ВМ.

```
sudo hostnamectl set-hostname k8s-master
```

2. Определяем имя хоста, для его дальнейшего использования вместо IP-адреса.

   Чтобы узнать адрес хоста вводим команду:

```
sudo ip addr show
```

Добавляем на сервер DNS соответствующие записи (на каждой ВМ необходимо добавить записи о каждом хосте.

```
sudo vi /etc/hosts
```

Пример записи:

```
10.0.2.6 k8s-master
```

3. Обновляем пакеты

```
sudo apt update && sudo apt upgrade
```

4. Выполняем установку необходимых пакетов

```
sudo apt install curl apt-transport-https git iptables-persistent
```

– curl — утилита для отправки GET, POST и других запросов на http-сервер. Понадобится для загрузки ключа репозитория Kubernetes.
– apt-transport-https — позволяет получить доступ к APT-репозиториям по протоколу https.

- git — утилита для работы с GIT. Понадобится для загрузки файлов из репозитория git.
- iptables-persistent — утилита для сохранения правил, созданных в iptables (не обязательна, но повышает удобство).

5. Отключаем файл подкачки для дальнейшей работы с Kubernetes (без выполнения данной команды Kubernetes не запустится)
   Для дальнейшей работы, чтобы swap не появился после перезагрузки сервера, открываем на редактирование файл:

```
sudo vi /etc/fstab
```

В файле комментируем данную строку:

```
#/swap.img      none    swap    sw      0       0
```

Для разового отключения swap выполняем команду:

```
sudo swapoff -a
```

6. Загружаем дополнительные параметры ядра

```
sudo vi /etc/modules-load.d/k8s.conf
```

Вводим данные:

```
br_netfilter
overlay
```

* модуль ***br_netfilter*** расширяет возможности netfilter; ***overlay*** необходим для Docker.

7. Загружаем модули в ядро:

```
sudo modprobe br_netfilter
```

```
sudo modprobe overlay
```

8. Проверяем, что данные модули работают:

```
sudo lsmod | egrep "br_netfilter|overlay"
```

9. Изменяем параметры ядра.

   Создаем конфигурационный файл:

```
sudo vi /etc/sysctl.d/k8s.conf
```

Вводим данные:

```
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
```

* ***net.bridge.bridge-nf-call-iptables*** *контролирует возможность обработки трафика через bridge в netfilter.*

10. Применяем параметры командой:

```
sudo sysctl --system
```

Задание №3 Создание набора правил

На мастер-ноде (Master)

1. Выполняем команду:

```
sudo iptables -I INPUT 1 -p tcp --match multiport --dports
6443,2379:2380,10250:10252 -j ACCEPT
```

* в данном примере мы открываем следующие порты:
6443 — подключение для управления (Kubernetes API).
2379:2380 — порты для взаимодействия мастера с воркерами (etcd server client API).
10250:10252 — работа с kubelet (соответственно API, scheduler, controller-manager).

2. Для сохранения правил выполняем команду:

```
sudo netfilter-persistent save
```

На рабочей ноде (Worker):

1. На нодах для контейнеров открываем такие порты:

```
sudo iptables -I INPUT 1 -p tcp --match multiport --dports 10250,30000:32767 -j
ACCEPT
```

* где:

10250 — подключение к kubelet API.
30000:32767 — рабочие порты по умолчанию для подключения к подам (NodePort Services).

2. Сохраняем правила командой:

```
sudo netfilter-persistent save
```

Задание №4 Установка и настройка Docker

1. Устанавливаем Docker относительно Лабораторной работы №1 на все ВМ
2. После установки разрешаем автозапуск сервиса docker:

```
sudo systemctl enable docker
```

3. Создаем файл:

```
sudo vi /etc/docker/daemon.json
```

Добавляем запись:

```
{
  "exec-opts": ["native.cgroupdriver=systemd"],
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "100m"
  },
  "storage-driver": "overlay2",
  "storage-opts": [
    "overlay2.override_kernel_check=true"
  ]
}
```

** для нас является важной настройкой **cgroupdriver** — она должна быть выставлена в значение **systemd**. В противном случае, при создании кластера Kubernetes выдаст предупреждение. Хоть на возможность работы последнего это не влияет, но мы постараемся выполнить развертывание без ошибок и предупреждений со стороны системы.*

4. Перезапускаем docker:

```
sudo systemctl restart docker
```

Задание №5 Установка Kubernetes

1. Установку необходимых компонентов выполним из репозитория. Добавим его ключ для цифровой подписи:

```
curl -s https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo apt-key
add -
```

2. Создадим файл с настройкой репозитория:

```
sudo vi /etc/apt/sources.list.d/kubernetes.list
```

Добавляем в файл запись:

```
deb https://apt.kubernetes.io/ kubernetes-xenial main
```

3. Обновим список пакетов:

```
sudo apt update
```

4. Устанавливаем пакеты:

```
sudo apt install kubelet kubeadm kubectl
```

* где:

kubelet — сервис, который запускается и работает на каждом узле кластера. Следит за работоспособностью подов.
kubeadm — утилита для управления кластером Kubernetes.
kubectl — утилита для отправки команд кластеру Kubernetes.

5. Нормальная работа кластера сильно зависит от версии установленных пакетов. Поэтому бесконтрольное их обновление может привести к потере работоспособности всей системы. Чтобы этого не произошло, запрещаем обновление установленных компонентов:

```
sudo apt-mark hold kubelet kubeadm kubectl
```

6. Установка завершена — можно запустить команду:

```
kubectl version --client --output=yaml
```

Задание №6 Создание кластера

Настройка control-plane (мастер ноды)

1. Выполняем команду на мастер ноде:

```
kubeadm init --pod-network-cidr=10.244.0.0/16
```

** данная команда выполнит начальную настройку и подготовку основного узла кластера. Ключ **--pod-network-cidr** задает адрес внутренней подсети для нашего кластера.*

2. В окружении пользователя создаем переменную KUBECONFIG, с помощью которой будет указан путь до файла конфигурации kubernetes:

```
export KUBECONFIG=/etc/kubernetes/admin.conf
```

3. Чтобы каждый раз при входе в систему не приходилось повторять данную команду, открываем файл:

```
sudo vi /etc/environment
```

Добавляем в файл строку:

```
export KUBECONFIG=/etc/kubernetes/admin.conf
```

4. Посмотреть список узлов кластера можно командой:

```
kubectl get nodes
```

5. Чтобы завершить настройку, необходимо установить CNI (Container Networking Interface)

```
kubectl apply -f
https://raw.githubusercontent.com/coreos/flannel/master/Documentation/kube-flannel.yml
```

Настройка worker (рабочей ноды)

1. Мы можем использовать команду для присоединения рабочего узла, которую мы получили после инициализации мастер ноды или вводим (на первом узле):

```
kubeadm token create --print-join-command
```

2. Данная команда покажет нам запрос на присоединения новой ноды к кластеру, например:

```
kubeadm join 192.168.0.15:6443 --token f7sihu.wmgzwxkvbr8500al \
    --discovery-token-ca-cert-hash
sha256:6746f66b2197ef496192c9e240b31275747734cf74057e04409c33b1ad280321
```

3. Копируем его и используем на двух наших узлах. После завершения работы команды, мы должны увидеть:

```
Run 'kubectl get nodes' on the control-plane to see this node join the cluster.
```

4. На мастер ноде вводим:

```
kubectl get nodes
```

Мы должны увидеть:

```
NAME                      STATUS      ROLES                  AGE   VERSION
k8s-master                 Ready       control-plane,master   18m   v1.20.2
k8s-worker1                Ready       <none>                 79s   v1.20.2
k8s-worker2                Ready       <none>                 77s   v1.20.2
```